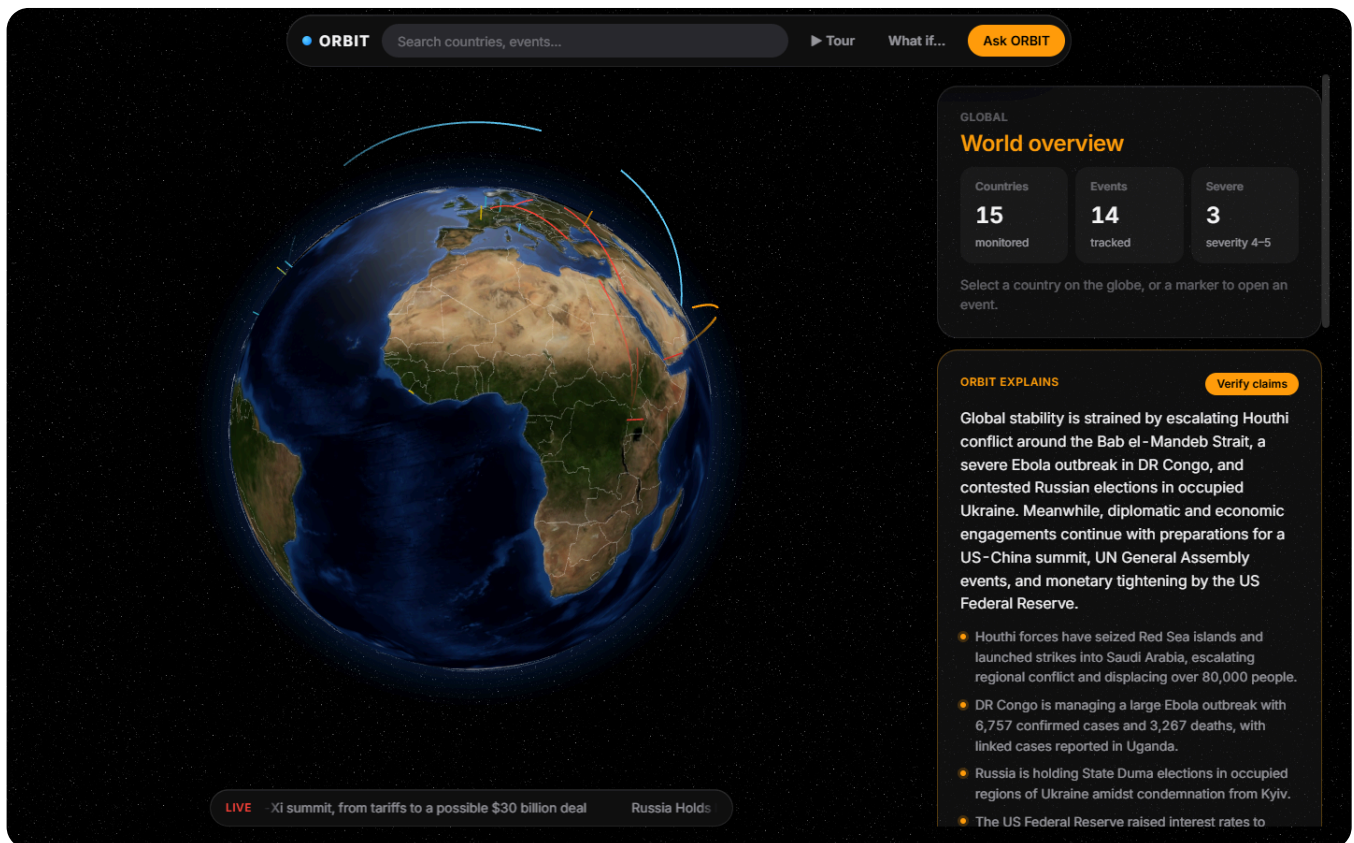




ORBIT

Review preparation: pitch, features, architecture, demo script, Q&A

AI-powered global intelligence on a 3D globe. Global → Country → Event → Explanation → Ask ORBIT.

Team: Arham · Ayman · Affan · Shrey · Hardik

Generated 2026-09-19 from the Markdown sources in the repository. If this PDF and the repository disagree, the repository wins.

ORBIT: review preparation

Everything you need for the review: what ORBIT is, what is special about it, how it works, how to demo it, and the questions a reviewer is likely to ask, with the answers. Figures are from the running app on 19 Sep 2026.

1. The 30-second pitch

ORBIT is an AI global-intelligence platform on a 3D globe. It turns scattered world events (conflicts, disease outbreaks, trade and policy moves) into one explorable picture: **Global → Country → Event → Explanation → Ask ORBIT**. Its edge is that it shows **how events ripple across the world** (a Red Sea attack → oil prices → India's import bill → the rupee and pump prices), lets you **simulate "what if"** (the Strait of Hormuz closes), and **checks its own AI with three independent verifiers**, so every claim is traceable to a source. It runs entirely on free services and keeps working with the wifi off.

One-liner: *"Google Earth meets a Bloomberg terminal, with an AI analyst that shows its sources and checks its own work."*

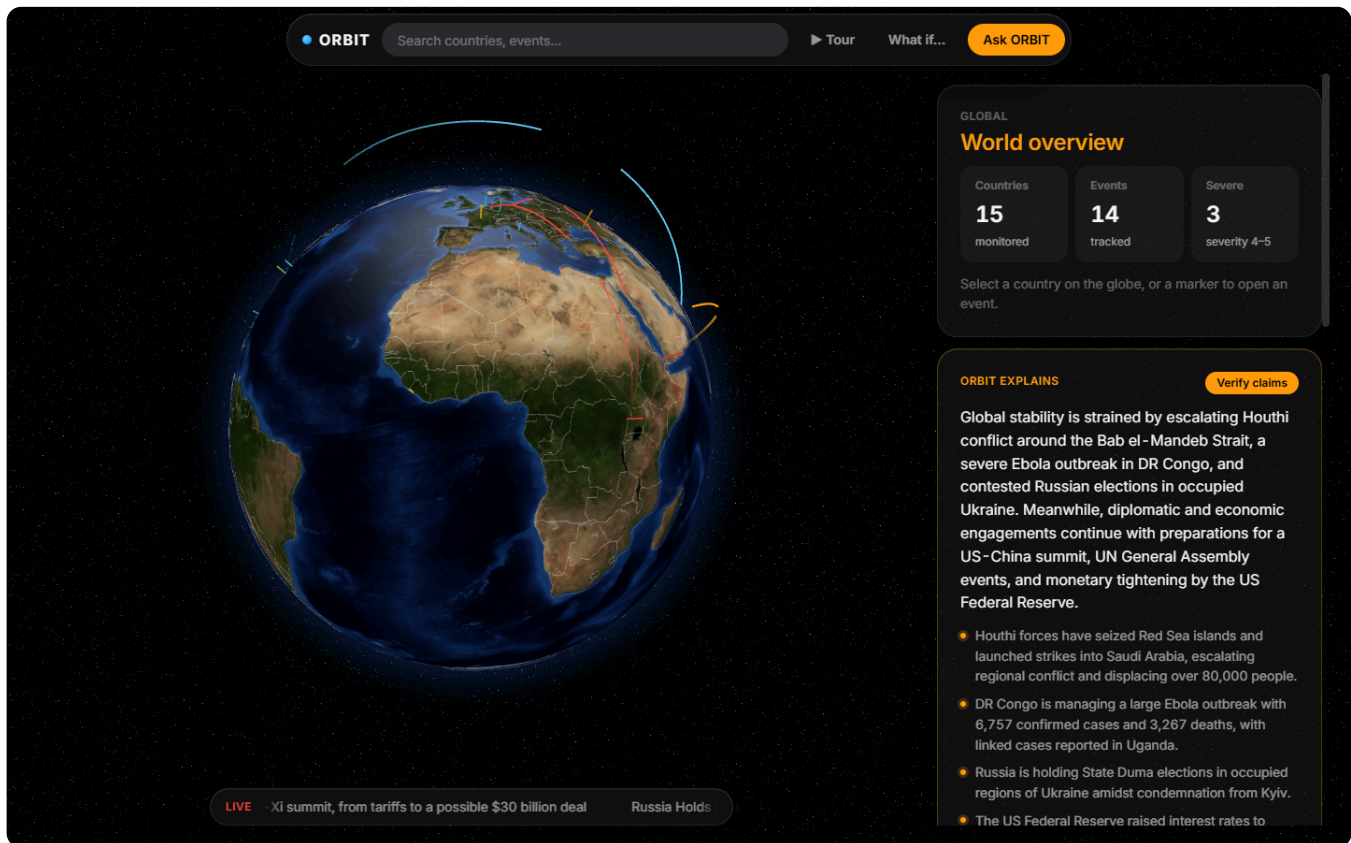
2. The problem

- World news is fragmented: a conflict, an outbreak and an oil-price move land in different apps and feeds.
- People see **events**, not **consequences**. The link from "Houthis seize Red Sea islands" to "petrol in Delhi" is invisible to most readers.
- AI summaries are fluent but **unverifiable**. Users can't tell sourced fact from model guesswork.

3. The solution

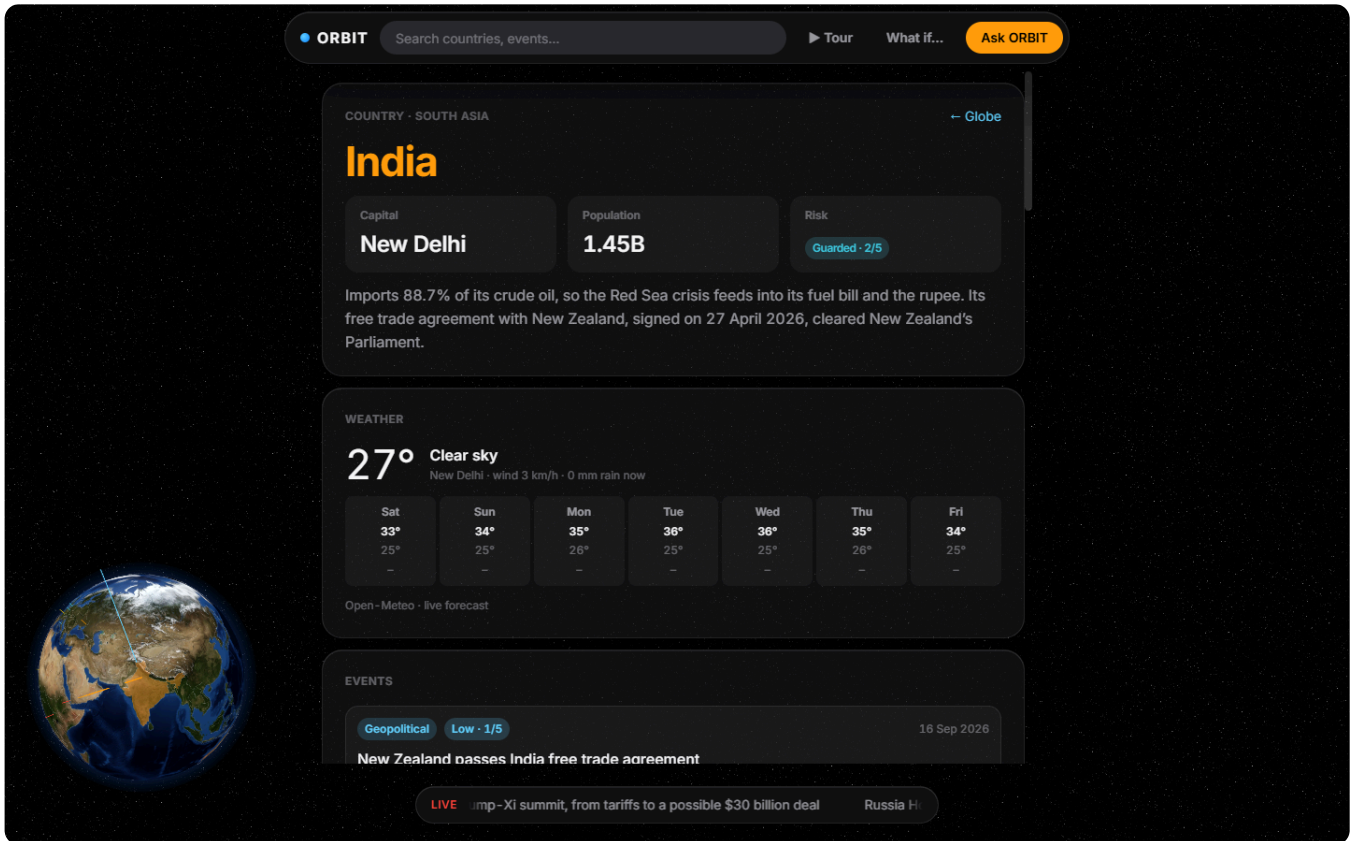
Need	ORBIT's answer
See the world at a glance	A realistic 3D globe with severity-coloured event pins, pulsing rings on severe events, and a live headline ticker
Understand one place	Country view: risk level, events, history timeline, live news, markets, weather, AI briefing
Understand one event	Event view: summary, sources, ripple effects, AI explanation with a claim-by-claim verification
See consequences	Ripple effects: sourced cause → effect chains, drawn as animated arcs on the globe
Explore scenarios	What if... simulator: chokepoint closures with transparent arithmetic
Ask anything	Ask ORBIT: a grounded chat that knows what you're looking at, by text or voice
Present it	Guided tour: the globe flies through the top stories with a natural narrator voice

4. Feature walkthrough



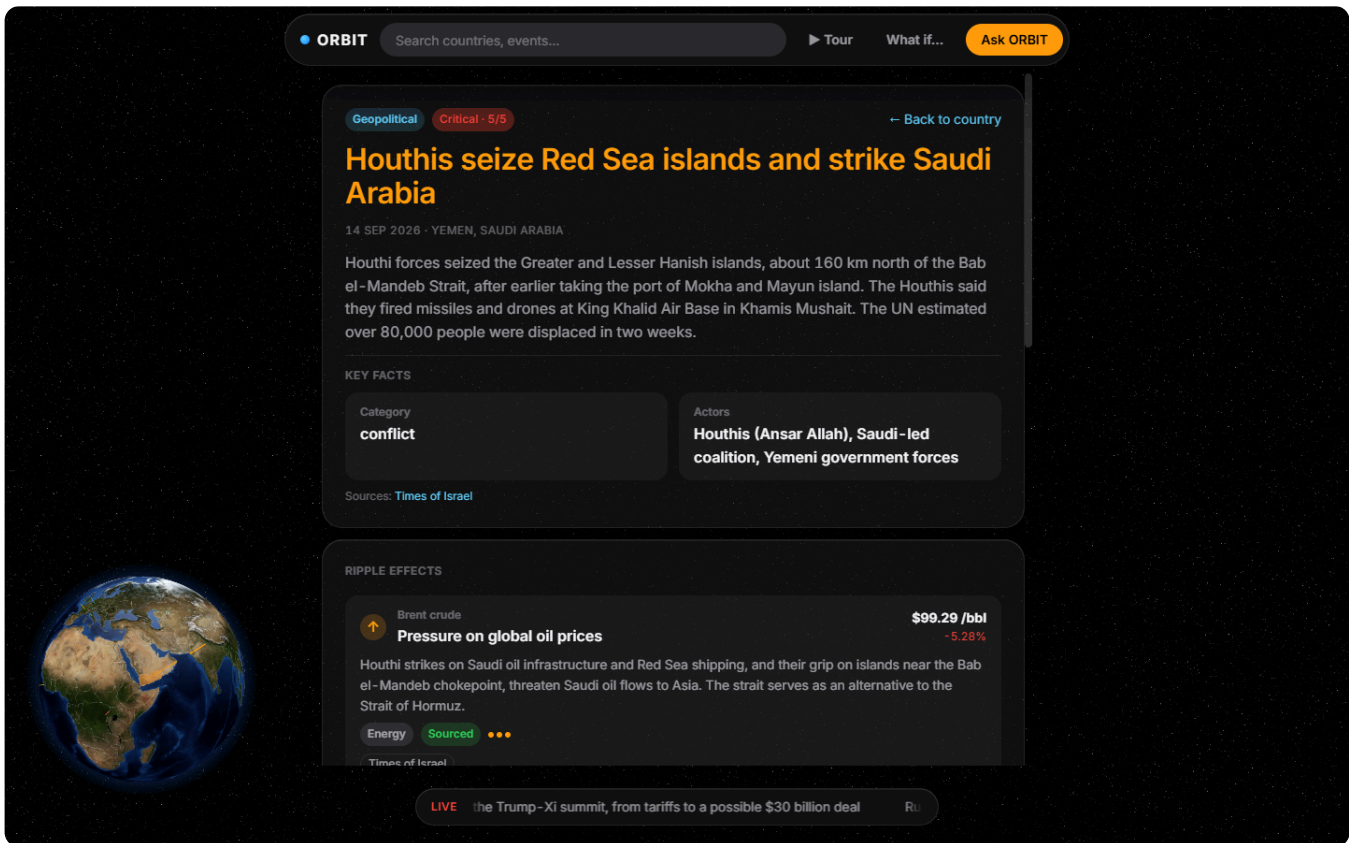
4.1 Global view

- A realistic Earth: NASA Blue Marble texture, terrain bump map, specular (shiny) oceans, atmosphere, starfield. It rotates slowly when idle and pauses while you drag.
- **14 curated real events** from 15 countries (Sept 2026): 7 health (WHO, UKHSA NaTHNaC) and 7 geopolitical (Red Sea / Houthis, Russia's Duma vote in occupied Ukraine, the Trump–Xi summit, the Fed rate decision, the Taçi verdict, the NZ–India FTA, UNGA81).
- Pins are coloured by severity (1–5). Severity 4–5 gets pulsing rings. Hover for a tooltip, click to open.
- **Ripple arcs**: animated arcs coloured by channel (energy, shipping, trade, finance, health, security).
- Right column: the AI world briefing, health signals, headline markets. A live headline ticker runs at the bottom.
- An Apple-style UI: frosted-glass panels, a capsule toolbar (search, ► Tour, What if..., Ask ORBIT), SF typography, rounded radii (8/14/22/28 px).



4.2 Country view (/country/IND)

- The globe shrinks into a mini globe (bottom-left) centred on the country. Click it to return to the world.
- Profile: capital, population, region, **risk level**, summary (e.g. India: 88.7% crude import dependence, from ThePrint).
- Events in and involving the country, a **history timeline** (key dates), **live news** (GDELT), **markets** (NIFTY 50, Sensex, Reliance, TCS, HDFC Bank, Infosys, USD/INR, Brent, Delhi petrol/diesel), with sparklines, **weather** for the capital (Open-Meteo), **ripple effects touching this country**, and an **AI country briefing**.



4.3 Event view and the AI explanation

- Event details, sources (each with publisher and reliability), and ripple effects caused by the event.
- **AI explanation** (Gemini): 2–3 sentence summary, key points, the sources used, and a confidence level.
- **Claim check** (our trust feature): the explanation is split into up to 6 claims, and each is checked **independently** by three verifiers:
 1. **Gemini** (Google): LLM judgement against ORBIT's data
 2. **Groq gpt-oss-120b** (a different model from a different company): an independent LLM judgement
 3. **Rules**: deterministic, no AI. Every number in the claim and most key terms must appear in ORBIT's data.
- Each claim shows supported / unsupported / contradicted per verifier, plus an **agreement** badge (agree / disagree) and an overall agreement score. In our live run: **6 of 6 claims, all three verifiers agreed**.

4.4 Ripple effects (USP #1)

- An **ImpactLink** connects an **event** to a **country or market**: channel (energy, shipping, trade, finance, health, security), direction (up / down / risk), strength 1–3, effect, mechanism, and **basis**:
 - **sourced** : the source states it
 - **analysis** : ORBIT's reasoning built on sourced facts, always **labelled "ORBIT analysis"** in the UI and by the AI.
- Links chain (`followsImpactId`): **Houthi seize Red Sea islands** → **Brent crude** ↑ → **India's crude import bill** ↑ → **rupee pressure / pump prices**. Each hop shows the live market value it touches (Brent \$99.29, USD/INR 95.88, Delhi petrol ₹102.12/l).
- 10 curated links. On the globe they are animated arcs, and Ask ORBIT can explain them.

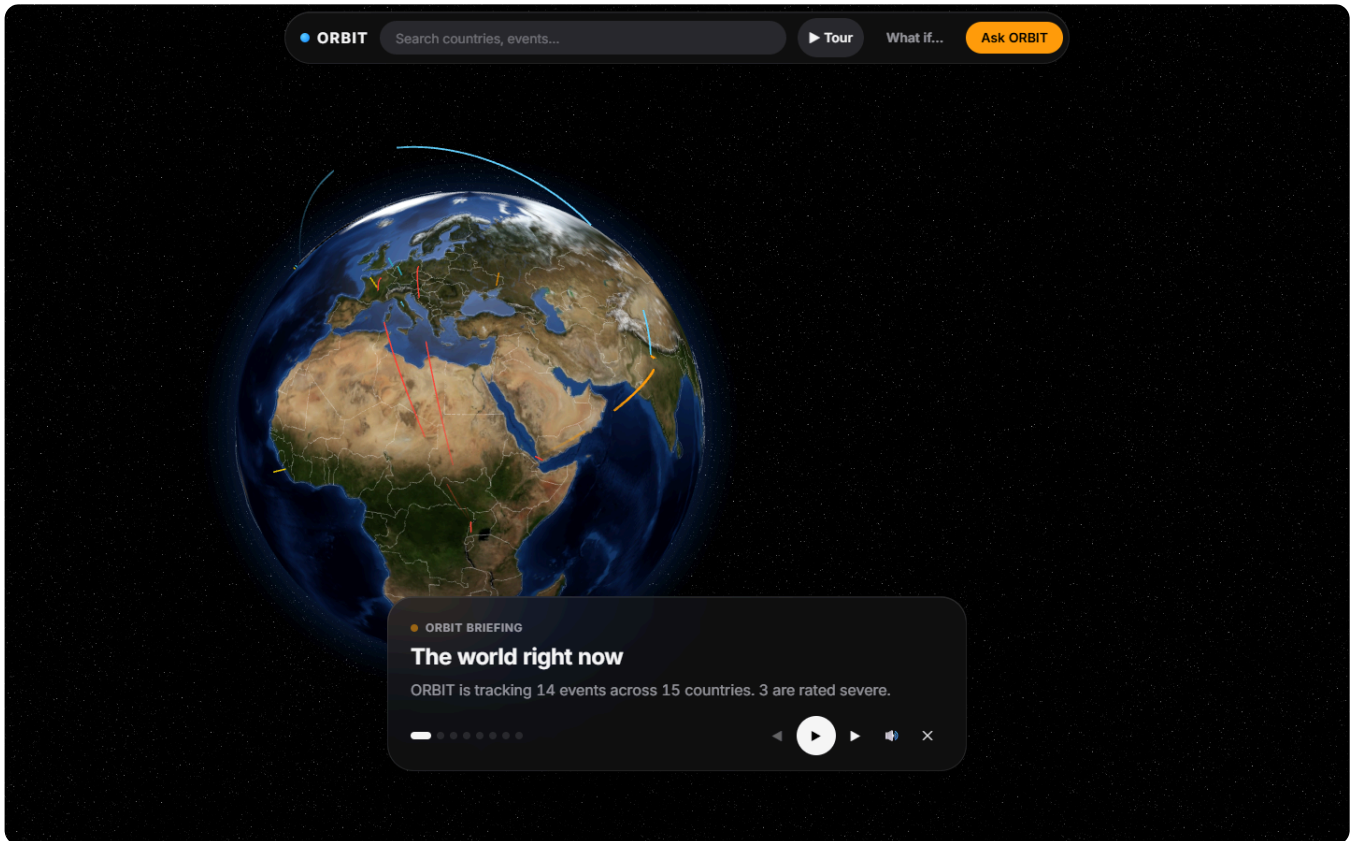


4.5 What if... simulator (USP #2)

- Scenarios: **Strait of Hormuz closes** and **Bab el-Mandeb shipping halts**. Each has sourced facts (U.S. EIA, Times of Israel).
- The user chooses the oil-price shock** (slider -30% to +100%, presets +10/+30/+60, marked illustrative). ORBIT **does not predict the shock**. It only does transparent arithmetic on live and sourced numbers.
- Every result card has a **"How this is calculated"** formula and source chips. A permanent badge reads **"Simulation, not a forecast"**.
- The globe flies to the chokepoint. Affected countries are tinted by role (importer orange, exporter cyan, transit red), and arcs run from the chokepoint to each importer.
- Ask ORBIT on this page knows the current simulation and always calls it a simulation.

Worked example: Hormuz, +30%:

Output	Formula	Result
New Brent price	live Brent $\times$ (1 + 30%) = 99.29 $\times$ 1.3	\$99.29 $\rightarrow$ \$129.08
Oil flow with no route out	Hormuz flow – pipeline bypass = 20 – 2.6 million b/d (EIA)	17.4 million b/d $\approx$ 17% of world use
India crude import bill	same volumes, price only	+30%
Crude cost per litre in India	Δ Brent $\times$ USD/INR $\div$ 158.987 l/bbl = 29.79 $\times$ 95.88 $\div$ 158.987	up to +₹18.0/l (if fully passed through, before taxes and subsidies; today's Delhi petrol ₹102.12)
Rupee pressure, market risk	qualitative, from the ripple graph	labelled ORBIT analysis



4.6 Guided tour with a narrator (USP #3, the demo opener)

- **Tour** in the toolbar, or open `/?tour=1`. Stops: intro → the 5 most severe events → the strongest ripple chain → outro.
- The camera makes a slow cinematic flight to each stop. Rotation pauses, the stop's country lights up, and the side panels fade away.
- Caption card: eyebrow, title, one-line briefing with its ripple line, progress dots, `< ▶/II >`, `🔊`, `✕` and **Open** (jump into that event). Keys: `←` `→` move, space pause, M mute, Esc exit.
- **Narrator voice: Gemini text-to-speech** (voice "Charon", directed as a calm documentary narrator). Clips are cached on disk, so each sentence is generated once, then plays instantly and offline. The next stop's clip is fetched while the current one plays. If there is no key or quota, the browser's own voice takes over.
- **Safety:** the narration text is built only from event summaries and sourced ripple links, never free AI text, so the tour cannot say anything unsourced.

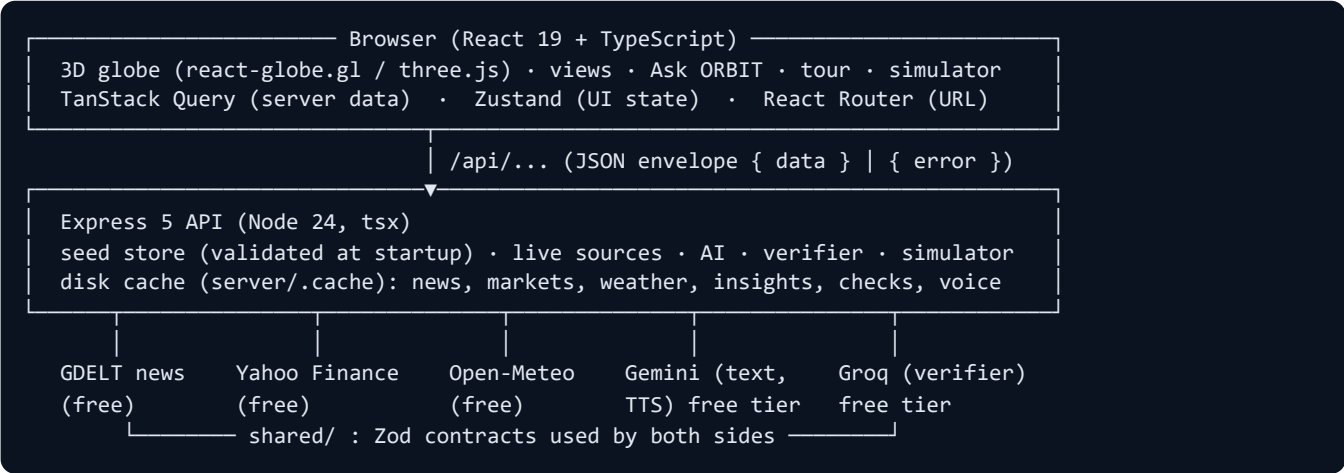
4.7 Ask ORBIT (chat)

- A drawer on the right. It knows the context (world, country, event, or the current simulation) and offers suggested questions for it.
- Answers are grounded (see §6), cite source chips, and show which provider answered (Gemini / Offline analysis).
- **Voice:** `🗣️` ask by voice (Chrome/Edge speech recognition; words appear as you speak and the question sends when you stop), and `🔊` **Listen** reads any answer aloud in the narrator voice.

4.8 Search and navigation

- Search countries and events from the toolbar. The URL is the state (`/country/IND/event/...`), so every view can be linked and the back button works.

5. Architecture



Why this design:

- **One contract, both sides:** every entity is a Zod schema in `shared/`. The server validates seed data and AI output against it, and the frontend gets the types, so a mismatch fails at build time, not on stage.
- **Keys stay on the server:** the browser never sees an API key (no `VITE_` secrets).
- **Seed first, live on top:** curated, fact-checked data always exists. Live sources enrich it and are cached. Any failure falls back to cache, then to seed, so the screen is never blank.
- **The URL is the state:** views can be shared and linked, and back and forward work.

Tech stack

Layer	Choice	Why
Frontend	Vite 8, React 19, TypeScript (strict)	Fast dev loop, the team knows React, types catch mistakes
Globe	react-globe.gl (three.js / WebGL)	Proven 3D globe with points, polygons, arcs and rings
Data layer	TanStack Query · Zustand · React Router	Caching and retries for free · tiny UI store · URL state
Styling	CSS Modules + design tokens	No runtime cost, scoped styles, one place for colours
API	Express 5 on Node 24 via tsx	Simple, TypeScript without a build step
Contracts	Zod 4	Runtime validation and static types from one definition
AI	Vercel AI SDK: Gemini (text + TTS), Groq	Structured output, provider-agnostic, free tiers
Tests	Vitest + Testing Library	Same config as Vite; tests the real API with fixtures

6. The AI, and why it can be trusted

1. **Grounding:** the model receives a compact JSON **context** (the countries, events, headlines, ripple links, markets and simulation relevant to the question) and strict rules: use only this context, cite source IDs exactly, say so when the context doesn't answer, label "ORBIT analysis", call simulations simulations, no speculation or advice.
2. **Structured output:** responses must match a Zod schema (summary, key points, sourceIds, confidence). Invalid output is rejected, and unknown source IDs are dropped.

3. **Independent verification:** Gemini + Groq + rules check every claim (\$4.3). Disagreement is shown, not hidden.
4. **Model fallback:** Gemini 3.5 Flash → 3.5 Flash-Lite. On any failure: the cached insight → the offline analyser (clearly labelled "Offline analysis").
5. **Cost and quota discipline:** insights are reused for 30 minutes, verification reports and voice clips are cached, and requests are retried once.

7. Data sources (all free, \$0)

Source	Used for	Mode
WHO Disease Outbreak News, UKHSA NaTHNaC	Health events (Ebola DRC, dengue, West Nile, Lassa...)	Curated seed
Federal Reserve, UN, Al Jazeera, Kyiv Post, Euronews, LN24, Times of Israel	Geopolitical events	Curated seed
U.S. EIA (Hormuz)	Simulator: 20 mb/d flow, 2.6 mb/d bypass	Curated seed
ThePrint, Business Today, Goodreturns	India crude dependence 88.7%, Delhi fuel prices	Curated seed
World Bank WDI	Population, country metadata	Curated seed
GDELT DOC 2.0	Live news per country	Live, cached
Yahoo Finance chart API	Brent, USD/INR, NIFTY, Sensex, stocks (every 15 min)	Live, cached
Open-Meteo	Capital-city weather	Live, cached
Natural Earth / NASA Blue Marble	Country shapes, Earth textures	Static files

17 sources in total. Each has a reliability rating, and every event, headline, link and market value points to one.

8. Reliability: it survives a live demo

- **Wifi off:** seed data plus the disk cache for news, markets, weather, insights, claim checks and voice clips. The demo keeps working. Where there's nothing cached, the UI says so plainly (e.g. "Weather is unavailable right now.").
- **API quota hit:** model fallback, then cache, then the offline analyser. The voice falls back to the browser's voice.
- **Rate limits:** GDELT requests are queued 10 s apart with a back-off, and refreshed in the background.
- **Startup validation:** bad seed data (a broken reference, wrong ID format) stops the server with a clear message.
- **Error boundaries** around the globe, each view, and Ask ORBIT: one failure never blanks the app.

9. Security and privacy

- API keys only in a local `.env` (gitignored), read on the server. Never in the frontend, commits or chat.

- All input is validated with Zod (IDs, query params, request bodies, text length limits).
- No user accounts and no personal data stored. Voice recognition runs in the browser.

10. Quality

- **123 automated tests** (25 files): contracts, seed-data integrity, every API route, AI grounding and fallbacks, verification, the simulator's arithmetic, the tour's stop order, voice UI (mocked), and frontend views.
- `npm run check` = lint + tests + typecheck + production build, required before every merge.
- Tests never touch real APIs or keys: they use fixtures, the mock AI and a temporary cache.

11. Team and process

Person	Role
Ayman	Drove the build session; country, event and timeline views
Hardik	Integrator: reviews and merges PRs, demo laptop
Affan	AI behaviour and tone, API keys
Arham	Visual direction, layout, globe look
Shrey	Featured countries and events, fact checks, demo rehearsal

We used an AI pair-programmer (Claude Code) to write the code, following a frozen architecture and contracts. Decisions were owned by the team, work was split into a task manifest (`tasks.json`), and every change went through a PR with tests. The docs (`README` , `docs/`) describe everything, so any teammate can run and explain it.

12. Demo script (about 5 minutes)

Time	Do	Say
0:00	Open <code>localhost:5173/?tour=1</code> , press ▶	"This is ORBIT. Let it brief you." (let 2 stops play)
0:45	At the Red Sea stop press Open	"Every event comes with sources, and ORBIT shows what happens next."
1:15	Scroll to ripple effects, then the AI explanation → Claim check	"Red Sea → Brent → India's import bill → the rupee. And the AI's claims are checked by three independent verifiers: Gemini, Groq, and a rule checker with no AI. They agree."
2:15	Click What if... → Hormuz, press +30%	"What if Hormuz closes? You choose the shock, and ORBIT shows the maths: Brent to \$129, 17 million barrels a day stranded, up to ₹18 a litre in India. A simulation, not a forecast."
3:15	Open Ask ORBIT → 🗣️ "Why would India be hit hardest?" → 🔊 Listen	"Ask by voice. The answer cites its sources, and it can read it aloud."
4:00	Click India on the globe	"Every country: live markets, weather, news, history, and an AI briefing."
4:30	Back to the globe	"All free data, all sourced, and it still works with the wifi off."

Before the demo: `DATA_MODE=live`, keys in `.env`, `npm run dev`, `npm run warm:voice`, open India, the Red Sea event and `/simulate` once (this caches them). Use Edge in full screen at 100% zoom, volume up.

Backup plan: if the internet fails, carry on: everything is cached. If the voice fails, press  to mute and narrate yourself (captions still advance every 9 s).

13. Likely questions, with answers

Q: Where does the data come from? Is it real? Yes. 14 curated events from Sept 2026 with 17 named sources (WHO, UKHSA, the Fed, EIA, Kyiv Post, Euronews...), plus live GDELT news, Yahoo Finance markets and Open-Meteo weather. Every item links to its source.

Q: How do you stop the AI hallucinating? Four layers: it only sees ORBIT's data (grounding), its output must match a schema with valid source IDs, three independent verifiers check each claim, and ORBIT's own analysis is always labelled. If the AI fails, we show a labelled offline analysis instead of guessing.

Q: Why three verifiers? Two LLMs from different companies make different mistakes, so agreement between them means more. The rule-based checker needs no AI and no internet: it proves the numbers are really in the data. Showing disagreement builds trust.

Q: Is the simulator a prediction? No, and it says so. The user picks the price shock. ORBIT only does arithmetic on sourced facts (EIA volumes) and live prices, and shows every formula. Qualitative effects are labelled as analysis.

Q: If there is a new headline tomorrow, does ORBIT add it by itself? Yes. Headlines, markets and weather have always refreshed themselves from live APIs. Since Review 2, events do too: ORBIT reads the live feed, classifies the headline, writes a one-sentence summary from the headline text only, validates it against the event contract and adds it to the globe as "auto-detected · unverified", kept separate from the curated events. "Scan now" runs it while you watch. Details and guardrails: `docs/AUTO_EVENTS.md`.

Q: How are ripple effects created? Could it scale? Today they are curated and sourced by the team, which keeps them accurate. The schema (event → target, channel, basis, chain) is ready for AI-proposed links that are then verified by the same claim checker. That is the next step.

Q: What does it cost to run? \$0. Gemini and Groq free tiers, keyless public data APIs. Caching keeps us inside the free quotas.

Q: What happens if the internet or an API goes down? The app keeps working from the seed data and the disk cache, and the AI falls back to cached or offline answers. We tested this with the wifi off.

Q: Why a 3D globe instead of a map? Global chains (Red Sea → India, Hormuz → East Asia) read naturally as arcs on a globe, and it makes geography intuitive. The globe shrinks to a mini-globe in detail views so the content stays readable.

Q: How is the narrator voice made? Gemini's text-to-speech model, directed to sound like a documentary narrator. Each sentence is generated once and cached, so the tour plays instantly and offline. Its script comes only from sourced data.

Q: Why TypeScript + Zod everywhere? One contract for both the frontend and the server, validated at runtime. With five people working in parallel over 24 hours, it catches mismatches before the demo does.

Q: How did you split the work / use AI to build it? We froze the architecture and contracts first, split the work into a task manifest with owners and acceptance tests, and used an AI pair-programmer for the implementation. Humans owned the decisions (design, data, AI tone, featured events) and reviewed every PR. 111 tests guard it.

Q: What would you build next? AI-proposed ripple links (verified before publishing), more scenarios (Suez, Malacca, Taiwan Strait), personalised alerts ("tell me when something affects my country or portfolio"), more countries and live event ingestion, and mobile.

Q: What are the limitations? Events and ripple links are curated (15 countries, 14 events), not yet auto-ingested. Free-tier quotas limit heavy use (caching mitigates this). Voice input needs Chrome or Edge. The simulator is deliberately simple arithmetic, not an economic model.

14. Numbers cheat sheet

Fact	Value
Countries / events / sources	15 / 14 (7 health, 7 geopolitical) / 17
Ripple links / market series / scenarios	10 / 10 / 2
Verifiers	3 (Gemini, Groq gpt-oss-120b, rules)
AI models	Gemini 3.5 Flash (+ Flash-Lite), Gemini 3.1 Flash TTS, Groq gpt-oss-120b
Tests	111 across 25 files
Cost	\$0 (free tiers, keyless public APIs)
Brent / USD-INR / Delhi petrol (snapshot)	\$99.29 / ₹95.88 / ₹102.12 per litre
Hormuz (EIA)	20 mb/d ≈ 20% of world use; 2.6 mb/d bypass; 84% of its crude goes to Asia
India crude import dependence	88.7% (ThePrint, Jul 2026)
Hormuz +30% (simulation)	Brent \$129.08 · 17.4 mb/d stranded · India bill +30% · up to +₹18.0/l

15. API at a glance

GET /api/health · /countries · /countries/:id · /countries/:id/events · /countries/:id/timeline · /events · /events/:id · /news · /sources · /insights/:type/:id · /verify/:type/:id · /impacts · /markets · /weather/:countryId · /scenarios · /simulate · POST /ask · POST /speech . Full contract: docs/API.md .

ORBIT

AI-powered global intelligence on an interactive 3D globe: geopolitical and health signals, news, history, markets, weather and explainable, multi-model-verified AI, in one flow: **Global** → **Country** → **Event** → **Explanation** → **Ask ORBIT**.



What makes ORBIT different

Feature	What it does
Ripple effects	Shows how an event spreads: Red Sea crisis → Brent crude → India's crude import bill → the rupee and pump prices. Each link is sourced or labelled "ORBIT analysis", has a live market figure, and is drawn as an animated arc on the globe.
What if... simulator	<code>/simulate</code> : close the Strait of Hormuz or Bab el-Mandeb, choose an oil-price shock (−30...+100%), and ORBIT computes the knock-on effects with visible formulas on live and sourced numbers. Labelled "Simulation, not a forecast".
Multi-model claim check	Every AI explanation is split into claims. Gemini, Groq (gpt-oss-120b) and a deterministic rule checker verify each claim against ORBIT's data independently, and the UI shows where they agree or disagree.
Guided tour with a narrator	► Tour (or <code>/?tour=1</code>): the globe flies through the most severe events and the strongest ripple chain while a Gemini voice narrates. Narration is built from sourced data only. Keys: ← → move, space pause, M mute, Esc exit.
Voice in Ask ORBIT	🗣 ask by voice (Chrome/Edge) and 🔊 Listen to any answer.

Feature	What it does
Grounded AI	Gemini answers only from ORBIT's data, cites source IDs, flags analysis and simulations, and falls back to cached insights, then to an offline analyser. It never shows a blank screen.
Automatic event detection	ORBIT reads today's live headlines, proposes new events itself and marks them auto-detected and unverified, separate from the curated ones. See docs/AUTO_EVENTS.md .
Live, free data	GDELT news, Yahoo Finance markets (Brent, USD/INR, NIFTY, Sensex, Indian blue chips), Open-Meteo weather. Every source is free and keyless, and every value is cached and has a seeded fallback for offline demos.
Realistic Earth, Apple-style UI	NASA Blue Marble textures, terrain, shiny oceans and stars; frosted-glass panels, capsule toolbar, SF typography.

All of it runs on free tiers (\$0) and still works with the wifi off.

Built in 24 hours by Arham, Ayman, Affan, Shrey and Hardik. This README is the team's starting point, and each section links to the document that holds the full detail.

Quick start

Requirements: **Node 22.12+** (team uses 24; see `.nvmrc`), npm, Git. Keep the repo **outside OneDrive** (e.g. `C:\dev\orbit`).

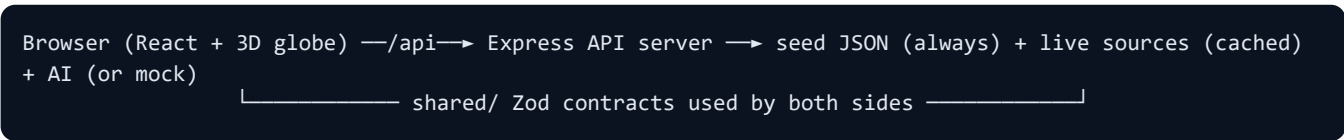
```
git clone https://github.com/Ayman-Alam-India/orbit C:\dev\orbit
cd C:\dev\orbit
npm install
copy .env.example .env      # optional: every value has a safe default
npm run dev                 # open http://localhost:5173
```

`npm run dev` starts two processes, `[web]` (Vite on `:5173`) and `[server]` (the API on `:8787`). The browser calls `/api/...` and Vite forwards it to the API server. Everything works offline with mock data and a mock AI.

Commands

Command	What it does
<code>npm run dev</code>	Web + API servers with hot reload
<code>npm run dev:web</code> / <code>npm run dev:api</code>	Only one of them
<code>npm test</code> / <code>npm run test:watch</code>	Vitest: contracts, seed data, API, frontend, tasks.json
<code>npm run lint</code>	ESLint
<code>npm run build</code>	Typecheck everything (<code>tsc -b</code>) + production frontend build
<code>npm run check</code>	lint + test + build. Must pass before every PR
<code>npm run format</code>	Prettier
<code>npm run docs:pdf</code>	Rebuild <code>docs/ORBIT-Playbook.pdf</code> and <code>docs/ORBIT-Review.pdf</code>
<code>npm run warm:voice</code>	Pre-generate the tour narration (dev server running). Run before the demo

Architecture



- **Frontend** (`src/`): Vite, React 19, TypeScript, react-globe.gl, TanStack Query, Zustand, CSS Modules + design tokens.
- **API** (`server/`): Express 5 run with tsx. It validates the seed data at startup, keeps keys server-side, and falls back to cache or seed when live data fails.
- **Contracts** (`shared/`): one Zod schema per entity (Country, OrbitEvent, NewsHeadline, Source, TimelineEvent, AllInsight, Ask, VerificationReport, ImpactLink, MarketQuote, WeatherReport, Scenario/SimulationResult).
- **AI** (`server/ai/`): Gemini via the Vercel AI SDK (model fallback list) with structured output. Insights are cached, and when Gemini fails ORBIT uses the cache, then the offline analyser. Claim verification: Gemini + Groq + rules (`server/ai/verify/`). Narrator voice: Gemini TTS, cached as WAV (`server/ai/tts.ts`).
- **Live sources** (`server/sources/`): GDELT (throttled queue), Yahoo Finance (refreshed every 15 min), Open-Meteo (cached 30 min). All go through the disk cache in `server/.cache/`.
- **Simulator** (`server/sim/simulate.ts`): a pure, unit-tested engine. Every output carries its formula and sources.

Details: [docs/PLAYBOOK.md](#).

Repository structure

<code>shared/</code>	Contracts (schemas + API routes)	<code>server/</code>	API server, seed data, AI, live
<code>sources</code>			
<code>src/</code>	Frontend (layout, globe, features, ui)	<code>public/data/</code>	Country shapes for the globe
<code>docs/</code>	Team documentation + PDF	<code>scripts/</code>	Tooling
<code>tasks.json</code>	Task manifest	<code>AGENTS.md</code>	Rules for people and coding
<code>agents</code>			
<code>mcp/, workshop-data/</code>	Hackathon organiser tooling: do not modify		

Full map: [docs/PLAYBOOK.md](#).

Shared contracts

Types come only from `@shared` (`import { type Country, API_ROUTES } from '@shared'`).

Rule	Value
Country IDs	ISO alpha-3, uppercase: <code>IND</code>
Other IDs	Prefix + lowercase slug: <code>evt_</code> , <code>hs_</code> , <code>news_</code> , <code>src_</code> , <code>tl_</code> , <code>ins_</code>
Dates	ISO 8601 UTC: <code>2026-09-19T10:00:00Z</code>
Coordinates	<code>{ lat, lng }</code>
Severity	Integer 1–5
Responses	<code>{ data }</code> or <code>{ error: { code, message } }</code>

API routes, entity fields and the mock-data format: [docs/API.md](#).

Environment variables

Defined in `.env.example` (committed) and parsed in `server/env.ts` . Put real values in a local `.env` , which is never committed.

Variable	Default	Meaning
PORT	8787	API server port
DATA_MODE	mock	mock = seed data only, offline · live = + live news, markets and weather (use live for the demo)
AI_PROVIDER	mock	mock or google
GOOGLE_GENERATIVE_AI_API_KEY	empty	Gemini key, only in your local <code>.env</code>
GOOGLE_MODEL	gemini-3.5-flash,gemini-3.5-flash-lite	Gemini model(s), tried in order
GROQ_API_KEY	empty	Groq key (free tier): the second model for claim verification
GROQ_MODEL	openai/gpt-oss-120b	Groq model used for verification
GOOGLE_TTS_MODEL	gemini-3.1-flash-tts-preview,gemini-2.5-flash-preview-tts	Narrator voice model(s), tried in order. No key = the browser's voice
GOOGLE_TTS_VOICE	Charon	Gemini prebuilt voice (Charon, Kore, Puck, Aoede...)

Secrets never go in code, commits, chat, or `VITE_` variables.

Team and ownership

Build model: Claude, running in Ayman's session, writes all the code. Each teammate is the **decision owner** for an area: Claude asks them the customization questions (layout, content, wording, interactions, data) before building, and they review and test the result. Teammates don't commit code; they comment on PRs or post in the chat. See [AGENTS.md](#).

Person	Decides and reviews
Hardik (integrator)	Reviews and merges every PR, sync points, the demo laptop
Affan	AI behaviour and tone (provides the AI key)
Arham	Visual direction, layout, globe look
Ayman	Drives the Claude session; country, event and timeline views
Shrey	Featured countries and events, fact checks, demo rehearsal

The folder-to-owner map and the shared files that need a separate PR are in [AGENTS.md](#).

Task system

All work is listed in `tasks.json`: ID, owner, priority, branch, allowed paths, dependencies, acceptance criteria, tests and handoff. To find out what to do, paste the prompt from `docs/ALLOCATOR.md` into your LLM with your name. Explanation of the fields and the dependency graph: `docs/PLAYBOOK.md`.

Git workflow

1. `git checkout main && git pull`, then `git checkout -b <branch from tasks.json>`
2. Small commits: `area: what changed`
3. `git pull origin main` and `npm run check`
4. PR titled `[ORB-XXX-NN] summary`. Hardik squash-merges.

Contract changes, conflicts and what never gets committed: `docs/INTEGRATION.md`.

Documentation

Document	For
AGENTS.md	Rules (read by Claude Code via <code>CLAUDE.md</code> , Codex, Cursor)
docs/PLAYBOOK.md	Everything in one place
docs/API.md	API and data contract
docs/INTEGRATION.md	Branches, PRs, contract changes
docs/DESIGN_SYSTEM.md	Tokens, primitives, UI conventions
docs/ALLOCATOR.md	Personal LLM task allocator
docs/ORB-Playbook.pdf	Printable reference (generated, the Markdown is the source)
docs/WALKTHROUGH.md · docs/AUTO_EVENTS.md	Explaining the code in a review, and how automatic detection works
docs/REVIEW.md · PDF	Review and judging prep: pitch, features, architecture, demo script, Q&A

Demo checklist

1. `.env` has `DATA_MODE=live`, `AI_PROVIDER=google`, the Gemini and Groq keys.
2. `npm run dev`, then `npm run warm:voice`. Open each featured country once so insights and claim checks are cached.
3. Use **Edge** in full screen. Start with `/?tour=1` and press ►. Then Red Sea event → claim check → What if... Hormuz +30% → Ask ORBIT by voice.
4. If the wifi dies, keep going: the cached data, AI answers and narration still work.

Credits

Country shapes: [Natural Earth](#) 1:110m admin-0 countries (public domain), via the [globe.gl](#) examples.

ORBIT API and data contract

Owner: Hardik. The code is the truth: schemas live in `shared/schemas/`, paths in `shared/api.ts`, and routes in `server/routes/`. This document explains them in plain words. If you find a difference, the code wins, so tell Hardik and this file gets fixed.

How it fits together



- In development, the browser calls `http://localhost:5173/api/...` and Vite forwards it to `http://localhost:8787`.
- The browser never calls external APIs or AI directly. Keys stay on the server.

Response format (every route)

Success, HTTP 200:

```
{ "data": <the thing you asked for> }
```

Failure, HTTP 4xx/5xx:

```
{ "error": { "code": "NOT_FOUND", "message": "Country \"XXX\" not found" } }
```

code	HTTP	Meaning
VALIDATION	400	Bad input: wrong ID format, invalid body, invalid query
NOT_FOUND	404	The ID is valid but does not exist, or the route does not exist
UPSTREAM	502	A live source failed and there was no fallback (should be rare)
INTERNAL	500	A bug on the server (check the server terminal for <code>[api] unexpected error</code>)

In the frontend, `apiGet / apiPost` (`src/api/client.ts`) unwrap `data` for you and throw an `ApiClientError` (`code`, `status`, `message`) on failure. If the server is not running you get `code: "NETWORK"`.

Routes

Always build paths with `API_ROUTES` from `@shared`. The column "Builder" shows how.

Method + path	Builder	Returns (data)	Errors
GET /api/health	<code>API_ROUTES.health</code>	<code>HealthStatus { status: "ok", dataMode, aiProvider }</code>	none
GET /api/countries	<code>API_ROUTES.countries</code>	<code>Country[]</code>	none
GET /api/countries/:id	<code>API_ROUTES.country(id)</code>	<code>Country</code>	400 bad ID, 404
GET /api/countries/:id/events	<code>API_ROUTES.countryEvents(id)</code>	<code>OrbitEvent[]</code> newest first, every event whose <code>countryIds</code> includes <code>id</code>	400, 404
GET /api/countries/:id/timeline	<code>API_ROUTES.countryTimeline(id)</code>	<code>TimelineEvent[]</code> oldest first	400, 404
GET /api/events?kind=	<code>API_ROUTES.events(kind?)</code>	<code>OrbitEvent[]</code> newest first. <code>kind</code> optional: <code>geopolitical</code> or <code>health</code>	400 bad kind
GET /api/events/auto	<code>API_ROUTES.autoEvents</code>	<code>OrbitEvent[]</code> ORBIT detected in live headlines itself (<code>origin: "auto"</code> , <code>unverified</code>). Empty in <code>DATA_MODE=mock</code>	none
POST /api/events/scan	<code>API_ROUTES.scanEvents</code>	<code>OrbitEvent[]</code> : runs a detection scan now instead of waiting for the background one	none
GET /api/events/:id	<code>API_ROUTES.event(id)</code>	<code>OrbitEvent</code>	400 bad ID, 404
GET /api/news?countryId=	<code>API_ROUTES.news(countryId?)</code>	<code>NewsHeadline[]</code> newest first. Without <code>countryId</code> : all	400 bad ID
GET /api/sources	<code>API_ROUTES.sources</code>	<code>Source[]</code>	none
GET /api/insights/:subjectType/:subjectId	<code>API_ROUTES.insight(type, id)</code>	<code>AIInsight</code> . <code>subjectType</code> = <code>global</code> (id <code>world</code>), <code>country</code> (country ID) or <code>event</code> (event ID)	400 bad type, 404 unknown subject
POST /api/ask	<code>API_ROUTES.ask</code>	<code>AskAnswer</code>	400 invalid body

Method + path	Builder	Returns (data)	Errors
POST /api/speech	API_ROUTES.speech	audio/wav (not JSON): the narrator voice for { text } (1–1500 chars). Gemini TTS, cached per sentence in server/.cache/speech	400 invalid body, 503 no key/quota (browser voice takes over)
GET /api/verify/:subjectType/:subjectId	API_ROUTES.verify(type, id)	VerificationReport : every claim of that insight checked by each verifier, with agreement	400 bad type, 404 unknown subject
GET /api/impacts?eventId=&countryId=	API_ROUTES.impacts(filter)	ImpactLink[] : ripple effects caused by an event, or touching a country (as cause, target or via its markets)	400 bad ID
GET /api/markets?countryId=	API_ROUTES.markets(countryId?)	MarketQuote[] : curated snapshot, live Yahoo Finance values in DATA_MODE=live	400 bad ID
GET /api/weather/:countryId	API_ROUTES.weather(id)	WeatherReport for the capital (Open-Meteo). 503 UPSTREAM when live data is off and nothing is cached	400, 404, 503
GET /api/scenarios	API_ROUTES.scenarios	Scenario[] : what-if chokepoint scenarios with sourced facts	none
GET /api/simulate?scenario=&brentPct=	API_ROUTES.simulate(id, pct)	SimulationResult : knock-on effects of a Brent shock (–30... +100%, clamped), each with formula and sources	400 bad input, 404 unknown scenario

POST /api/ask body (AskRequest):

```
{
  "question": "Why is dengue rising?",
  "context": { "countryId": "IND", "eventId": "hs_ind_dengue_surge" }
}
```

question is 1–500 characters. context and both of its fields are optional. With eventId, ORBIT answers about that event. With only countryId, it answers about that country. With neither, it answers about the world.

Shared rules for all data

Rule	Example
Country ID = ISO 3166-1 alpha-3, uppercase	IND, USA, BRA (matches <code>properties.id</code> in <code>public/data/countries.geojson</code> ; Kosovo is KOS)
Other IDs = prefix + lowercase slug (a-z, 0-9, _)	evt_usa_tariff_review, hs_ind_dengue_surge, news_..., src_..., tl_..., ins_...
Dates = ISO 8601 UTC with Z	2026-09-19T10:00:00Z
Coordinates = { lat, lng }	{ "lat": 28.61, "lng": 77.21 }
Severity = integer 1–5	1 Low, 2 Guarded, 3 Elevated, 4 High, 5 Critical
Field names = camelCase. Enum values = lowercase	occurredAt, "geopolitical"

Entities (data dictionary)

Country (`shared/schemas/country.ts`)

Field	Type	Notes
id	CountryId	IND
name	string	Display name
region	string	e.g. South Asia
capital	string	
centroid	{lat,lng}	Where the globe camera flies to
population	positive integer	
summary	string	One or two sentences
riskLevel	1–5	Overall current risk

OrbitEvent (`shared/schemas/event.ts`)

Either a **GeopoliticalEvent** (`kind: "geopolitical"`, ID `evt_...`) or a **HealthSignal** (`kind: "health"`, ID `hs_...`). Check `kind` before using kind-specific fields.

Field	Type	Notes
kind	geopolitical health	Decides which extra fields exist
id	evt_... or hs_...	Prefix must match <code>kind</code>
title, summary	string	
countryIds	CountryId[] (≥ 1)	The first one is the primary country (used for links)
location	{lat,lng}	Marker position
occurredAt	ISO date-time	

Field	Type	Notes
severity	1–5	
sourceIds	SourceId[]	Must exist in <code>sources.json</code>
tags	string[]	Free-form
geopolitical only: category	string	Free text until the category list is agreed
geopolitical only: actors	string[]	Governments, organisations, people
health only: indicator	string	e.g. <code>Dengue cases</code>
health only: metric (optional)	<code>{ value: number, unit: string }</code>	e.g. <code>{ 18400, "cases this month" }</code>

NewsHeadline (`shared/schemas/news.ts`)

`id` (`news_...`), `title`, `url`, `sourceId`, `publishedAt`, `countryIds` (may be empty), `eventId` (optional, links to an `OrbitEvent`), `publisher` (optional: the outlet's domain, e.g. `reuters.com`, set for live GDELT headlines whose `sourceId` is `src_gdelt`).

Source (`shared/schemas/source.ts`)

`id` (`src_...`), `name`, `url`, `type` (`news` | `government` | `international_org` | `research` | `dataset`), `reliability` (`high` | `medium` | `low`).

TimelineEvent (`shared/schemas/timeline.ts`)

`id` (`tl_...`), `countryId`, `date`, `datePrecision` (`day` | `month` | `year`), `title`, `description`, `eventId` (optional). A year-only fact is stored as 1 January of that year with `datePrecision: "year"` .

AllInsight (`shared/schemas/insight.ts`)

`id` (`ins_...`), `subjectType` (`global` | `country` | `event`), `subjectId` (`world`, a country ID or an event ID), `summary`, `keyPoints` (≥ 1), `sourceIds`, `confidence` (`low` | `medium` | `high`), `provider` (`mock`, `google`, ...), `generatedAt` . It is explainable because it always says which sources it used, how sure it is, and which AI produced it.

AskRequest / AskAnswer (`shared/schemas/ask.ts`)

Request: `question`, optional `context` { `countryId?`, `eventId?` }. Answer: `answer`, `sourceIds`, `provider`, `generatedAt` .

VerificationReport (`shared/schemas/verification.ts`)

The insight for a subject is split into claims (summary sentences + key points, max 6). Each claim is checked **independently** by every verifier against ORBIT's data: `gemini` and `groq` (LLMs, when their key is set) and `rules` (deterministic: every figure and most key terms must appear in the data; always runs, works offline).

Fields: `subjectType`, `subjectId`, `insightProvider`, `models` (verifiers that answered), `unavailable` ({ `model`, `reason` } for missing keys or failures), `claims[]` (`id` `clm_N`, `text`, `verdicts[]` of { `model`, `verdict`, `sourceIds`, `reason` }, `agreement`: `agree` / `disagree` / `single`), `agreementScore` (share of multi-verifier claims where all agree, 0–1), `generatedAt` . Verdicts: `supported`, `unsupported` (not in the data), `contradicted` . A complete report is cached; if a model fails later (e.g. no wifi), the cached report is served.

Data files (`server/data/seed/` and `server/data/fixtures/`)

One JSON file per entity, each a plain **array** of objects matching the schema above:

File	Contains	Owner
<code>countries.json</code>	<code>Country[]</code>	Shrey
<code>events.json</code>	<code>OrbitEvent[]</code> (both kinds mixed, each with <code>kind</code>)	Shrey
<code>news.json</code>	<code>NewsHeadline[]</code>	Shrey
<code>sources.json</code>	<code>Source[]</code>	Shrey
<code>timeline.json</code>	<code>TimelineEvent[]</code>	Shrey

Curated seed (what the app shows): 15 countries, 14 real events (7 geopolitical, 7 health signals), 7 headlines and 11 timeline entries, current as of 19 September 2026. Sourcing rules:

- Every fact comes from a page that was actually opened. Each `sources.json` entry points to that exact page (WHO Disease Outbreak News, the UKHSA NaTHNaC outbreak list, the Federal Reserve, the UN, and named news articles).
- Populations (2024) and capitals come from the World Bank (`SP.POP.TOTL` and its country API), and `centroid` is the capital's location. Kosovo uses the map's `KOS` code (the World Bank uses `XXK`).
- `riskLevel` and event `severity` are **ORBIT's editorial ratings** based on the tracked events, not an official index.
- Headlines use the article's real title and URL. Dates are publication dates (time set to 00:00 UTC).

Test fixtures (`server/data/fixtures/`): a small, stable, clearly fake data set (India, US, Brazil with `example.org` sources). Tests always run against it (`ORBIT_SEED_DIR` in `vitest.config.ts`), so curating the real seed never breaks a test.

To check your edits, run `npm test` . The seed test validates both data sets, checks that every country exists on the globe map, and rejects placeholder (`example.org`) sources in the curated seed. It prints the exact file, item and field that is wrong.

Data modes and future live data

Setting	mock (default)	live
<code>DATA_MODE</code>	Seed data only, fully offline	Seed data + live sources (<code>server/sources/</code>)
<code>AI_PROVIDER</code>	Offline analysis built from seed data	<code>google</code> (Gemini). Falls back to mock on any failure

Rules for every live source (server-side only):

1. Fetch → validate with the shared schema (drop invalid items) → save with `writeCache(key, data)` .
2. Merge with seed data: seed is always the base, live adds to it.
3. On any failure: `log [sources] ...` and return `readCache(key)` , then seed. Never throw to the route.
4. Live items must follow the same ID rules (e.g. `news_<source>_<slug>`) and reference existing sources.
5. A new external API key = a new empty line in `.env.example` + a field in `server/env.ts` .

Live news (implemented): GDELT (`server/sources/gdelt.ts`), free and keyless.

- Feeds: one per country (query = the country's name) plus a global feed (the countries with severity 4–5 events). English-language articles from the last 7 days, max 10 per feed, `id = news_gdeIt_<hash of URL>`.
- `GET /api/news` answers **immediately** with seed + cached live headlines. A stale or missing cache (over 30 min old) starts a background refresh, so GDELT's slowness never blocks the UI. Live headlines therefore appear on the next load.
- In `DATA_MODE=live` the server pre-fetches every feed on startup (`[sources] warming live news...`), one request every 10 s with a 20 s back-off on HTTP 429, because GDELT rate-limits hard. Start it well before the demo so the cache is full.
- Cached feeds live in `server/.cache/news-<COUNTRY|global>.json`. With wifi off, cached and seed headlines keep working.

Changing this contract

Adding a field, route or entity is a contract change. Follow `docs/INTEGRATION.md#contract-changes`. Hardik updates `shared/`, this document and the tests in one PR, which merges before the feature that needs it.

ORBIT design system

Owner: Arham. The tokens live in `src/styles/tokens.css` (the only place raw values may appear). The primitives live in `src/ui/`.

Direction (updated 2026-09-19): Apple-style shapes and layout. Frosted translucent materials, hairline borders, large rounded corners, capsule controls with tinted fills, San Francisco typography, soft depth instead of neon glow. Colours stay ORBIT's: orange highlights, cyan accents, in Apple system tints. A floating toolbar capsule sits at the top, and headlines run in a floating "Live" pill at the bottom. Effects stay sleek, not busy. Futuristic, cinematic, clean, like an intelligence centre. The globe is the hero, and panels float over it as dark glass.

Tokens

Colour

Token	Use
<code>--color-bg</code> , <code>--color-bg-elevated</code>	Page background, raised areas
<code>--color-surface</code> , <code>--color-surface-hover</code>	Glass panels (semi-transparent over the globe)
<code>--color-border</code> , <code>--color-border-strong</code>	Hairlines, focused/active outlines
<code>--color-text</code> , <code>--color-text-muted</code> , <code>--color-text-faint</code>	Primary, secondary, tertiary text
<code>--color-primary</code> , <code>--color-primary-dim</code>	Electric cyan: links, focus, geopolitical
<code>--color-accent</code>	Amber: Ask ORBIT, health, highlights
<code>--color-alert</code>	Orange: critical, errors
<code>--color-highlight</code>	Orange: panel titles, selected items
<code>--color-success</code> , <code>--color-danger</code>	Status only
<code>--severity-1</code> ... <code>--severity-5</code>	The shared 1–5 severity scale (cyan → amber → orange)
<code>--kind-geopolitical</code> , <code>--kind-health</code>	Event kind colours
<code>--globe-*</code>	Orange globe: ocean, land, hover, selected, borders, atmosphere, ring

Typography

Token	Value
<code>--font-display</code>	SF Pro Display (Apple) → Inter → Segoe UI
<code>--font-body</code> , <code>--font-label</code>	SF Pro Text (Apple) → Inter → Segoe UI
<code>--font-mono</code>	SF Mono → JetBrains Mono (code only)

Token	Value
<code>--text-xs ... --text-3xl</code>	0.75rem → 2.75rem scale
<code>--weight-regular/medium/bold</code>	400 / 500 / 700
<code>--tracking-label</code>	Letter spacing for uppercase eyebrow labels
<code>--leading-tight</code> , <code>--leading-body</code>	Line heights

Apple devices use San Francisco. Elsewhere Inter loads from Google Fonts, and offline it falls back to Segoe UI.

Spacing, borders, radii

- Spacing on a 4px grid: `--space-1` (4px) ... `--space-8` (64px).
- `--border-width` (1px). Radii: `--radius-sm` 8px, `--radius-md` 14px (tiles, rows), `--radius-lg` 22px (panels), `--radius-xl` 28px, `--radius-pill`.

Glass panels

`--glass-bg` (gradient fill), `--glass-border`, `--glass-highlight` (inner top highlight), `--glass-accent-border` (amber variant). Panels are dark glass: translucent, blurred, thin cyan hairline, 14px radius.

Shadows and glows

`--shadow-panel` (panel depth), `--glow-primary` (cyan), `--glow-accent` (amber), `--glow-alert` (orange), `--glow-soft` (panel hover), `--blur-panel` (glass blur). Use glows sparingly: for focus, hover and severity 5 only.

Motion and layers

- `--duration-fast/base/slow` (150/250/600ms) with `--ease-out`. `--duration-layout` (700ms) with `--ease-inout` for the globe squeeze between the global and detail views. They all become 0 when the user prefers reduced motion.
- `--z-globe` < `--z-overlay` < `--z-panel` < `--z-modal`.
- Layout sizes: `--panel-width` (420px, right column), `--center-width` (760px, detail view), `--mini-globe-size` (220px), `--drawer-width` (420px, Ask ORBIT drawer), `--topbar-height` (56px).

Globe

Realistic Earth (decided 2026-09-19): NASA Blue Marble day texture, terrain bump map, shiny oceans (specular water map) and a starfield page background. Textures are public domain (via the globe.gl examples) and live in `public/textures/` so the demo works offline. Countries are faint white hairlines. Hover lightens a country, and the selected country is filled orange (`--globe-land-selected`), which keeps orange as the highlight colour.

- **Pins:** every event gets a thin pin coloured by severity (`--severity-N`). More severe events get taller pins, but pins always stay short.
- **Rings:** only severity 4–5 events get a small ripple ring that fades as it grows (`src/globe/globeStyle.ts`, `-globe-ring`).
- **Motion:** the global view rotates slowly when idle. Rotation pauses while the user drags and resumes after 4 s. In detail views the globe becomes the bottom-left mini globe. In the global view it is centred left of the panel column.

- **Ripple arcs:** animated dashed arcs from an event to each country it affects, coloured by channel (`--channel-energy` orange, `--channel-finance` cyan, `--channel-health` red, `--channel-trade` green, `--channel-shipping` blue, `--channel-security` yellow). In detail views only the arcs touching the selected country remain.
- **Tooltips:** `.globe-tooltip` in `global.css` (the globe library renders labels as HTML strings, and the text is escaped).

Primitives (`src/ui` , `import from ../../ui`)

Component	Use
<code>Panel</code>	Every overlay section (glass). Props: <code>eyebrow</code> , <code>title</code> , <code>actions</code> , <code>tone</code> (<code>default</code> or <code>accent</code> for AI content), <code>children</code>
<code>QueryState</code>	Renders loading / error / data for a TanStack query: <code><QueryState query={q} label="country">{(data) => ...}</QueryState></code>
<code>Loader</code>	Loading state (<code>role="status"</code>)
<code>ErrorState</code>	Error state with optional retry (<code>role="alert"</code>)
<code>ErrorBoundary</code>	Wrap each feature section so a crash stays contained
<code>ItemList</code>	Vertical list with dividers and a required empty state
<code>SeverityBadge</code>	Shows severity 1–5 with its colour and label (severity 5 pulses slowly)
<code>Button</code>	Variants <code>primary</code> (cyan), <code>accent</code> (amber: Ask ORBIT, AI), <code>ghost</code> . Sizes <code>sm</code> , <code>md</code> . <code>iconOnly</code> requires <code>aria-label</code>
<code>Tag</code>	Small uppercase pill. <code>kind</code> (<code>geopolitical</code> / <code>health</code>) colours it; without <code>kind</code> it is neutral
<code>StatTile</code>	One labelled figure (<code>label</code> , <code>value</code> , optional <code>hint</code>). Put several in a grid for a stat strip

New primitives are decided with Arham (the design decision owner).

Common UI states (every data view needs all four)

State	How
Loading	<code>QueryState</code> / <code>Loader</code>
Error	<code>QueryState</code> / <code>ErrorState</code> (with retry)
Empty	<code>ItemList</code> <code>empty</code> text, or a sentence in muted text
Success	The content

Conventions

- One `Component.module.css` next to each component. Class names are camelCase (`styles.askButton`).
- Only tokens: `color: var(--color-text-muted)` , never `#8aa0b8` . The same goes for spacing, radii and shadows.
- Canvas/WebGL (globe): read tokens with `cssVar('--globe-land')` and `severityColor(severity)` from `src/styles/cssVar.ts` .

- Animations: define `@keyframes` inside the component's own `.module.css` (CSS Modules rename keyframes, so global ones can't be referenced).
- Text: sentence case. Eyebrow labels are uppercase via CSS, not typed in capitals.
- Accessibility: visible focus (global `:focus-visible` ring), `aria-label` on icon-only buttons, and colour never the only signal (severity badges also show the label and number).
- Numbers: `toLocaleString()` for large values. Dates: show them at their `datePrecision`.